BCSE303L
OPERATING SYSTEMS

Module 1 Lecture 3

Operating Systems Design Issues

Specifying and designing an operating system is a highly creative task.

1. Start the design by defining goals and specifications
2. Choice of hardware
3. System types
4. Specify the requirements
   - *User goals – operating system should be convenient to use, easy to learn, reliable, safe, and fast*
   - *System goals – operating system should be easy to design, implement, and maintain, as well as flexible, reliable, error-free, and efficient*
5. Separation of policy from mechanism

Operating Systems Design Issues

Specifying and designing an operating system is a highly creative task.

5. Separation of policy from mechanism

- *Mechanisms determine how to do something, policies decide what will be done.*
- *The separation of policy and mechanism is important for flexibility*
- *Policies are likely to change across places or over time, and each change in policy would require a change in the underlying mechanism*
- *Microkernel-based operating systems allows more advanced mechanisms and policies to be added via user-created kernel modules or user programs.*
- *Policy decisions are important for all resource allocation*

Operating Systems Design Issues

Specifying and designing an operating system is a highly creative task.

6. Implementation

- *Early operating systems were written in assembly language. Now, most are written in higher-level languages such as C or C++ with small amounts of the system written in assembly language.*
- *For example: Android kernel is written in C with some assembly language code; system libraries are written in C or C++, and its application frameworks in Java*
- *The **advantages** of using a higher-level language the code can be written faster, more compact, easier to understand and debug, and easier to port to other hardware.*
- ***Disadvantages**: reduced speed and increased storage requirements*

Abstractions

Abstractions are a way to represent complex systems or components in a simplified way, exposing only essential features and hiding implementation details.

Using abstractions, Operating System provides a more comprehensible and manageable interface for users and developers, while hiding the intricate details of the underlying system.

Some example abstractions are:

Process Abstraction: Represents a process as a single entity, hiding details like memory layout, registers, and execution state.

File Abstraction: Represents files as a sequence of bytes, hiding storage details like disk blocks and inode structures (metadata).

Socket Abstraction: Represents network connections as a socket, hiding details like protocol specifics and network hardware.

Device Abstraction: Represents hardware devices as a standardized interface, hiding details like device registers and interrupts.

Abstractions

Abstraction provides,

- *Simplification: Hide complexity, making it easier to understand and interact with the system.*
- *Modularity: Enable separate development and maintenance of components.*
- *Portability: Make it easier to move code between different platforms or architectures.*
- *Security: Limit access to sensitive information, improving security*

Abstraction can be achieved through,

- *Encapsulation: Hiding implementation details within a module or component.*
- *Interfaces: Providing a standardized way to interact with a component or system.*
- *APIs: Offering a set of functions or services to access a system or component.*

Processes in Operating System

A process in an operating system is essentially a program in action/execution.

- *It's the active version of a program that's currently running instructions.*
- *A process is the fundamental unit of work for the operating system.*

Some Process Types –

1. *User Processes: Execute user programs.*
2. *System Processes: Perform OS functions, like process management.*
3. *Daemon Processes: Run in the background, providing services.*

Resources in Operating System

- *It refers to the components or facilities that are managed and allocated to processes or programs to enable them to execute and complete their tasks.*
- *Essential components that programs need to run.*
- *The OS acts as resource manager, allocating these resources fairly and efficiently among all the running programs.*
- *The Operating System manages resources by doing the following: allocation, deallocation, scheduling, protection and sharing.*

Resources in Operating System - categories

- Physical Resources
  - CPU (Central Processing Unit)
  - Memory (Main Memory or RAM)
  - I/O Devices (Keyboard, Mouse, Printer, etc.)
  - Storage Devices (Hard Disk, Solid State Drive, etc.)
- Virtual Resources
  - Virtual Memory (Swap Space or Page File)
  - Virtual CPUs (in a virtualized environment)
- Software Resources
  - Files and Directories
  - I/O Streams (Input/Output Channels)
  - Sockets (Network Connections)
  - Semaphores (Synchronization Mechanisms)
- Abstract Resources
  - Processes (Threads, Tasks, or Jobs)
  - Threads (Lightweight Processes)
  - Synchronization Objects (Mutexes, Locks, etc.)

Security in Operating System

Security in an Operating System (OS) is crucial to protect the system, its resources, and user data from unauthorized access, malicious activities, and potential threats.

Why Security?

- *Protect user data and privacy*
- *Prevent unauthorized system access*
- *Ensure the integrity of data and programs*
- *Maintain system availability and reliability*
- *Comply with security regulations and standards*
- *Support secure communication and transactions*
- *Provide a trusted platform for applications and services*

Security in Operating System

Some of the security design issues are:

1. **Access Control:** The OS manages access to resources, such as files, directories, and devices, through mechanisms like *permissions, access control lists (ACLs), and attribute-based access control (ABAC).*
2. **Authentication:** The OS verifies the identity of users and processes through authentication mechanisms like *passwords, biometrics, and Kerberos*.
3. **Authorization:** The OS determines what actions a user or process can perform on a resource, based on their *role, privileges, and permissions*.
4. **Encryption:** The OS provides encryption mechanisms, like *file encryption and secure sockets layer (SSL)*, to protect data from unauthorized access.
5. **Integrity:** The OS ensures the *integrity of data and programs* by preventing unauthorized modifications or tampering.

Security in Operating System

Some of the design issues are:

6. Confidentiality: The OS protects sensitive information from unauthorized access, use, or disclosure.
7. Availability: The OS ensures that resources and services are available to authorized users when needed.
8. Threat Protection: The OS provides protection against various threats, such as *malware, viruses, Trojan horses, spyware, and ransomware*.
9. Vulnerability Management: The OS patch management and vulnerability scanning help identify and remediate vulnerabilities.
10. Auditing and Logging: The OS provides auditing and logging mechanisms to track security-related events and monitor system activity.

Networking in Operating System

- Networking refers to the ability of an Operating System to connect to and communicate with other devices, systems, or networks.
  - Resource Sharing: Sharing files, printers, and other resources between devices.
  - Communication: Exchanging data, messages, or requests between devices or systems.
  - Remote Access: Accessing remote systems or networks, like VPNs or remote desktops.

Multimedia in Operating System

Multimedia refers to the ability of an Operating System to handle and manage various forms of media, such as:

- Audio: Playing, recording, and manipulating audio files.
- Video: Playing, recording, and manipulating video files.
- Graphics: Rendering and manipulating graphical content.
- Images: Handling and manipulating image files.

Multimedia in Operating System

This involves

- Device Drivers: Managing device drivers for multimedia hardware, like sound cards or graphics cards.
- Media Formats: Supporting various media formats, like MP3, AVI, or JPEG.
- Media Players: Providing media player applications or frameworks.
- Multimedia APIs: Offering APIs for developers to access multimedia functionality.

Multimedia in Operating System

- Multimedia data includes continuous media in the form of audio or video files and conventional files.
- Multimedia *demands a very high data rate* and real-time delivery of the data. For any video display, *a rate of 24 to 30 frames per second* is necessary for the video to appear smooth to the human eye.
- The Multimedia API is a *collection of low-level APIs* that support flexible application development.
- Multimedia API can provide following applications: Video encode and decode, Video convert, Image and video processing, Object detection and classification, etc.

References

1. **Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, 2018, 10th Edition, Wiley, United States.**